

Coleta de dados no Youtube

Especialização em Ciência de Dados

Profa. Helen de Cássia Sousa da Costa Lima



Universidade Federal
de Ouro Preto

Roteiro

1. Criação de chave para uso da API
2. Entendimento do *script* de Coleta de dados no Youtube
3. Geração de uma nuvem de palavras com os comentários dos vídeos

Criação da chave

Acesse o [Console de APIs do Google](#). Se necessário, aceite os termos do serviço:

Acesso rápido

 API APIs e serviços

 IAM e admin

 Faturamento

 Compute Engine

 Cloud Storage

 BigQuery

 Rede VPC

 Kubernetes Engine

Criação da chave

Clique no botão “Criar projeto”

Digite o nome do projeto e clique em “Criar”

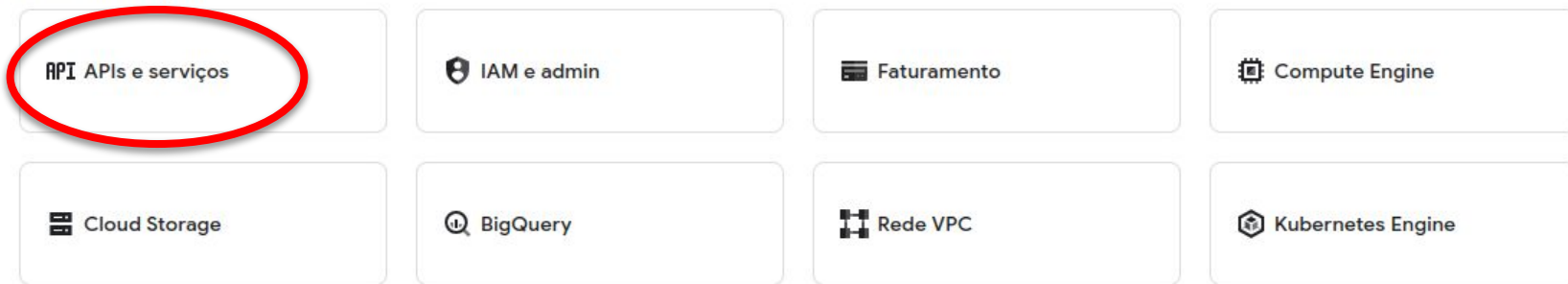


Você está trabalhando em [ufop.edu.br](#) > [MyProjectYoutube1](#)

Criação da chave

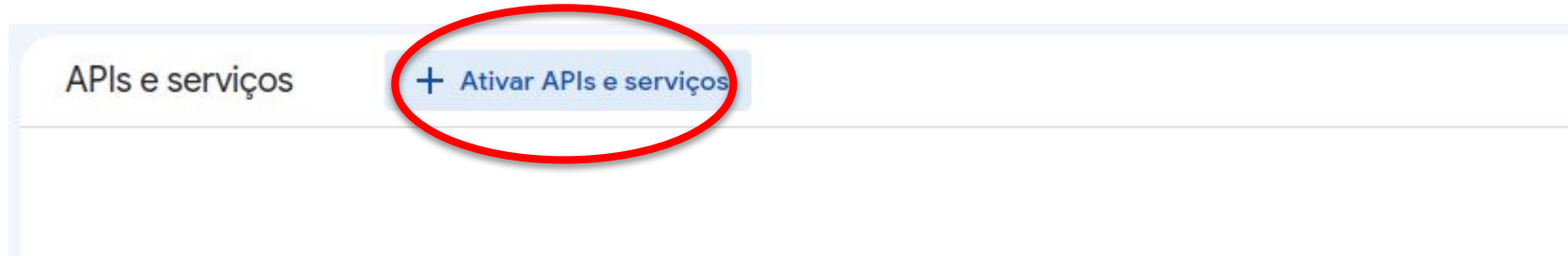
No acesso rápido, clique em “APIs e serviços”

Acesso rápido



Criação da chave

Clique em “Ativar APIs e serviços”



Criação da chave

Clique em “Youtube Data API v3”

YouTube

[Ver tudo \(3\)](#)



YouTube Data API v3

Google

The YouTube Data API v3 is an API that provides access to YouTube data, such as videos, playlists,...



YouTube Analytics API

Google

Retrieves your YouTube Analytics data.



YouTube Reporting API

Google

Schedules reporting jobs containing your YouTube Analytics data and downloads the resulting bulk...

Criação da chave

Clique em “Ativar”. No meu caso, já está ativa. Em seguida, vá em “Gerenciar”.



YouTube Data API v3

[Google](#)

The YouTube Data API v3 is an API that provides access to YouTube data, such as videos, playlists,...

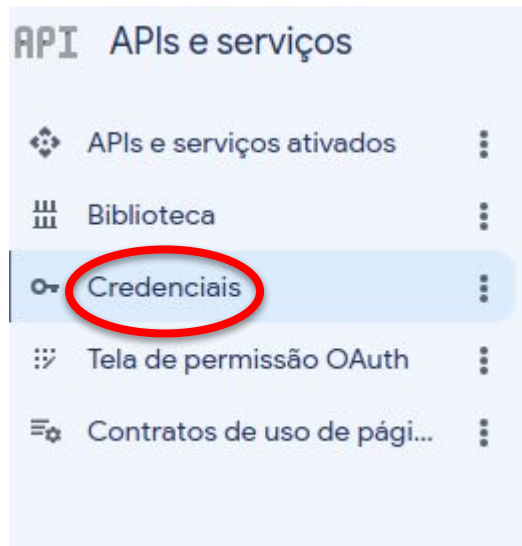
Gerenciar

Testar esta API [↗](#)

✓ API ativada

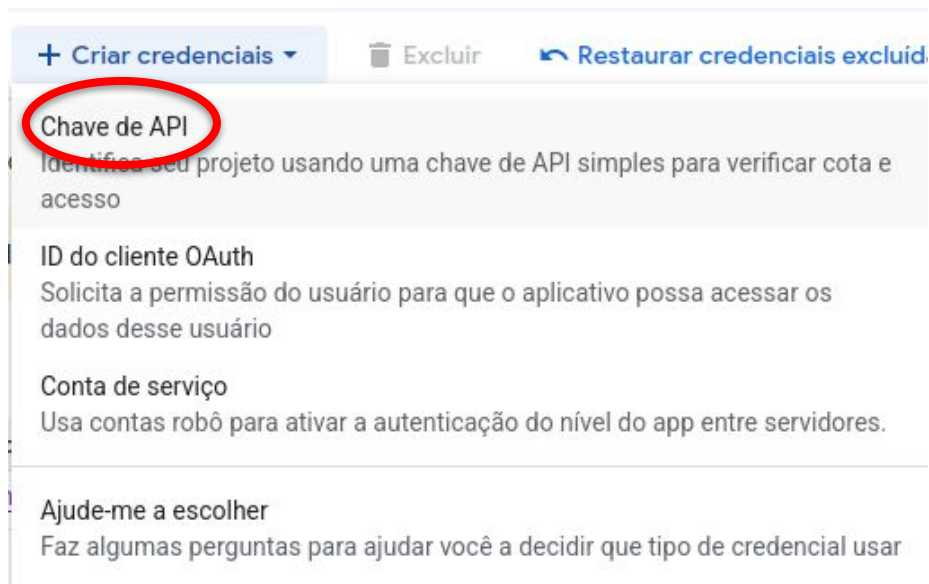
Criação da chave

Clique em “Credenciais”.



Criação da chave

Clique em “Criar credenciais”. E depois, “Chave de API”.



Criação da chave

Selecione a API “Youtube Data API v3”. E depois, “Criar”.

Criar chave de API



Nome *

Chave de API 2

Use um nome exclusivo para identificar sua chave de API

APIs que podem ser acessadas usando esta chave 



Criação de novas chaves: para usar as APIs Agent Platform (Vertex) ou Gemini com uma chave de API, vincule a chave a uma conta de serviço ao selecionar "Autenticar chamadas de API por uma conta de serviço" antes de aplicar a restrição de API.

Observação: uma [política da organização](#) rege a criação de chaves de API vinculadas a uma conta de serviço para projetos de uma organização do Google Cloud. [Saiba mais](#)

Selecionar restrições da API *

1 API

Esta lista inclui apenas as APIs ativadas neste projeto. Se você não encontrar a API necessária, [acesse a Biblioteca de APIs](#) para ativá-la.

APIs selecionadas:

YouTube Data API v3



Autenticar chamadas de API por uma conta de serviço

As chamadas de API feitas por essa chave serão autenticadas pela conta de serviço vinculada. Essa opção é necessária para acessar algumas APIs, como a Agent Platform (Vertex).

Restrinja sua chave para reduzir os riscos de segurança

Selecione um tipo de aplicativo específico para restringir o acesso

Restrições do aplicativo 



Nenhum



Sites

Criação da chave

Para acessar sua chave, basta clicar em “Exibir chave”.

Chaves de API

☐	Nome	Data da criação ↓	Restrições	
☐	Chave de API 1	26 de dez. de 2023	YouTube Data API v3 ...	Exibir chave ⋮

Fluxo de execução dos arquivos de coleta

Inicialização

Defina as configurações da sua pesquisa no arquivo *config.py*

Rode o arquivo *reset.py* no diretório pai que contém o diretório *crawler* para criar os arquivos iniciais

Execute: *python crawler/reset.py*

Execute o arquivo *script.py* no diretório pai que contém o diretório *crawler*
(*python crawler/script.py*)

Fluxo de execução dos arquivos de coleta

Zerar a pesquisa

Defina as configurações da sua pesquisa no arquivo *config.py*

Rode o arquivo *reset.py* no diretório pai que contém o diretório *crawler* para criar os arquivos iniciais

Execute: *python crawler/reset.py*

Execute o arquivo *script.py* no diretório pai que contém o diretório *crawler*

(*python crawler/script.py*)

Os dados da pesquisa anterior serão mantidos no diretório “files - [data da finalização]”

Configuração dos arquivos de coleta

config.py

Período de Coleta

```
# A coleta ocorre da data final para a  
data inicial
```

```
"start_date": [2023, 1, 1],  
"end_date": [2026, 5, 31]
```

Formato: [ano, mês, dia]

Palavras-chave no título do vídeo

```
# Filtro para títulos dos vídeos
```

```
"key words": [  
    "fastback", "abarth",  
    "limited edition",  
    "impetus", "audace"  
]
```

Configuração dos arquivos de coleta

config.py

Chaves da API YouTube

```
# KEYS da API v3 do YouTube
```

```
"youtube keys": [  
# "key 1",  
# "key 2",  
# "key 3"  
]
```

Queries de Pesquisa

```
# Queries que serão utilizadas na  
pesquisa
```

```
"queries": [  
"fastback avaliação 2023",  
"fastback avaliação 2024",  
"fastback avaliação 2025",  
"fastback avaliação 2026"  
]
```

Configuração dos arquivos de coleta

reset.py

Criação de Diretório

```
# Cria o diretório "files"
os.makedirs("files")
```

Gestão de Pastas Existentes

```
# Renomear pasta se já existir
if os.path.isdir("files"):
    os.rename("files", f"files
{datetime.now().strftime('%Y%m%d_%H%M%S')}")
createEmptyDir()
```

Inicialização de atual_date.csv

```
# Arquivo inicia com data final
with open("files/atual_date.csv", "w") as csvfile:
    fieldnames = ["year", "month", "day"]
    writer = csv.DictWriter(csvfile, fieldnames)

    writer.writerow({
        "year": config["end date"][0],
        "month": config["end date"][1],
        "day": config["end date"][2]
    })
```

Script de coleta - YouTubeAPIManager

script.py

Padrão Singleton: Gerenciamento de Instância Única

```
# O Singleton garante uma única instância para gerenciar chaves, cotas e conexões com a API
```

```
@staticmethod
def get_instance () -> "YouTubeAPIManager" :
    if YouTubeAPIManager.static YouTubeAPIManager == None:
        YouTubeAPIManager.static YouTubeAPIManager = YouTubeAPIManager ()
    return YouTubeAPIManager.static YouTubeAPIManager
```

Script de coleta - YouTubeAPIManager

script.py

Gerenciamento de Cotas e Troca de Chaves

```
# Recupera chaves e gerencia rotação/espera por quota excedida

def get_new_youtube_client(self):
    self.DEVELOPER_KEYS = config['youtube keys']
    if self.current_key_index >= len(self.DEVELOPER_KEYS) - 1:
        timeout = secondsUntil(5)
        log("key", f"Todas as chaves excederam a quota. Aguardando {timeout}s")
        GlobalState.get_instance().set_state("status", "sleeping")
        time.sleep(timeout) # Aguarda reset da quota
        GlobalState.get_instance().set_state("status", "working")
        self.current_key_index = 0
    else:
        self.current_key_index += 1 # Próxima chave
```

Script de coleta - YouTubeAPIManager

script.py

Log de Requisições (CSV)

```
# Salva log em CSV
def make_api_request(self, func, **kwargs):
    with open("requisições.csv", mode='a') as f:
        fields = kwargs.keys()
        w = csv.DictWriter(f, fieldnames=fields)
        w.writeheader() if f.tell() == 0 else None
        w.writerow(kwargs)
```

Tratamento de Erros e Exceções

```
# Gestão de erros de API
if razao == "quotaExceeded":
    log("key", "Quota excedida, trocando...")
    self.yt = self.get_new_client()
    log("key", f"Chave: {idx+1}/{len(KEYS)}")
elif e.resp.status == 403:
    print("Acesso restrito ao vídeo...")
```

Script de coleta - Detalhes do vídeo

script.py

Função get_video_details() - Estrutura dos dados

```
# Extrai os detalhes do vídeo a partir do ID
def get_video_details(video_id):

    details = {
        "video_id": video_id,
        "title": snippet.get('title'),
        "description": snippet.get('description'),
        "channel_id": snippet.get('channelId'),
        "published_at": snippet.get('publishedAt'),
        "tags": snippet.get('tags', []),
        "view_count": int(statistics.get('viewCount', 0)),
        "duration": contentDetails.get('duration'),
        "caption": contentDetails.get('caption') == 'true',
        ...
    }
```

Script de coleta - Comentários

script.py

Função get_comments()

```
# Extrai os comentários do vídeo a partir do ID
def get_comments(video_id, video_title, total_comment_count):
    try:
        method_func = lambda client, **kwargs: api_manager.youtube.commentThreads().list(**kwargs)
        print(">> Request de comentários" )

        # Realiza a requisição de comentários de 100 em 100
        response = api_manager.make_api_request(
            method_func, part="snippet,replies",
            videoId=video_id, maxResults=100,
            pageToken=page_token, textFormat="plainText"
        )
```

Script de coleta - Comentários

script.py

Função `get_comments()` - Estrutura de Dados

```
# Extrai os comentários do vídeo a partir do ID
def get_comments(video_id, video_title, total_comment_count):

    comments_data.append({
        "video_id": video_id, "comment_id": comment_id,
        "author": comment_info.get("authorDisplayName"),
        "author_profile_image_url": comment_info.get("authorProfileImageUrl"),
        "author_channel_url": comment_info.get("authorChannelUrl"),
        "author_channel_id": comment_info.get("authorChannelId", {}).get("value"),
        "comment": comment_info.get("textDisplay"),
        "published_at": comment_info.get("publishedAt"),
        "updated_at": comment_info.get("updatedAt", None), # Opcional
        "like_count": comment_info.get("likeCount"),
        "viewer_rating": comment_info.get("viewerRating", ""),
        "can_rate": comment_info.get("canRate", ""),
        "is_reply": False, "parent_id": None
    })
```

Script de coleta - *Replies*

script.py

Função `get_replies()` - Estrutura de Dados

```
# Extrai os comentários replies a partir do ID do vídeo e do ID do comentário pai
def get_replies(video_id, comment_id):
    replies_data.append({
        "video_id": video_id, "comment_id": item["id"],
        "author": reply_info.get("authorDisplayName"),
        "author_profile_image_url": reply_info.get("authorProfileImageUrl"),
        "author_channel_id": reply_info.get("authorChannelId", {}).get("value"),
        "comment": reply_info.get("textDisplay"),
        "published_at": reply_info.get("publishedAt"),
        "like_count": reply_info.get("likeCount"),
        "is_reply": True, "parent_id": comment_id
    })
```

Script de coleta - Detalhes do canal

script.py

Estrutura de Dados - JSON

```
# Detalhes extraídos do canal
details = {
    "channel_id": channel_id,
    "title": snippet.get('title'),
    "view_count": int(statistics),
    "subscriber_count": int(stats),
    "video_count": int(stats),
    "country": snippet.get('country', ""),
    "is_verified": branding.get(...),
    "keywords": branding.get(...),
    "profile_picture_url": url
}
```

Campos Principais Coletados:

- **Identificação:** ID, Título e Descrição básica do canal.
- **Métricas:** Visualizações totais, inscritos e total de vídeos postados.
- **Metadados:** Data de criação, país de origem e palavras-chave.
- **Status:** Verificação do canal e URL da imagem de perfil em alta resolução.

i A função `get_channel_details()` organiza esses dados para facilitar a análise posterior.

Script de coleta - Processamento do vídeo

script.py

Código-fonte: process_video()

```
# Recupera detalhes e processa comentários
def process_video(v_id, v_title, processed):
    details = get_video_details(v_id)
    ...
    chan = get_channel_details(details['cid'])
    ...
    comments = get_comments(v_id, v_title, count)
    ...

# Registra no histórico
processed_videos.add(v_id)
with open('processed_videos.csv', 'a') as f:
    writer = csv.writer(file)
    writer.writerow([video_id])
```

Fluxo de Operação:

- **1. Extração:** Obtém metadados básicos do vídeo via ID.
- **2. Contexto:** Captura informações do canal proprietário.
- **3. Interação:** Coleta comentários e respostas se dentro do limite.
- **4. Persistência:** Adiciona ID ao arquivo CSV de controle.

💡 Evita duplicidade ao consultar a lista de vídeos já processados.

Script de coleta - Busca pela pesquisa

script.py

Função `make_search_request()`

```
# Requisição de busca na API v3
def make_search_request(query, p_after, p_before, R_CODE, R_LANG):
    api_manager = YouTubeAPIManager.get_instance()
    method_func = lambda c, **k: api_manager.youtube.search().list(**k)
    search_response = api_manager.make_api_request(
        method_func, part="id,snippet", q=query,
        maxResults=10, type="video", order="relevance",
        publishedAfter=p_after, publishedBefore=p_before,
        regionCode=R_CODE, relevanceLanguage=R_LANG
    )
    n_videos = len(search_response.get('items', []))
    print(f"Retornou {n_videos} vídeos.")
```

Parâmetros de Busca:

- **Query:** Termo pesquisado.
- **Filtros:** Data (After/Before).
- **Local:** Region e Language.
- **Ordem:** Relevância.

🔍 A requisição retorna metadados básicos de até 10 vídeos por padrão.

Script de coleta - Função principal

script.py

Código-fonte: `main()`

```
# Configuração do período de coleta
# Carrega data do CSV de controle
df_atual_date = pd.read_csv( 'files/atual_date.csv' )
atual_date = {
    "year": df_atual_date.iloc[ 0, 0],
    "month": df_atual_date.iloc[ 0, 1],
    "day": df_atual_date.iloc[ 0, 2],
}

# Define objetos datetime
start_date = datetime(config['start_date'][0],
config['start_date'][1], config['start_date'][2])
end_date = datetime(atual_date["year"], atual_date["month"],
atual_date["day"])
```

Lógica Principal:

1. Persistência:

Lê o arquivo 'atual_date.csv' para retomar do ponto exato.

2. Intervalo:

Converte strings e números em objetos datetime comparáveis.

3. Controle:

Garante que a coleta respeite as datas de início e fim configuradas.

⚙ Define o escopo temporal da extração de dados.

Script de coleta - Função principal

script.py

Função `main()`: Carregamento de vídeos processados

```
# Cria um conjunto para IDs já processados
processed_videos = set()

# Tenta carregar dados do arquivo CSV
try:
    with open('files/processed_videos.csv', 'r') as file:
        reader = csv.reader(file)
        processed_videos = {row[0] for row in reader}
except
FileNotFoundError:
    pass # Mantém conjunto vazio
```

Gestão de Progresso:

- **Evitar Duplicidade:**
Garante que vídeos já coletados não sejam baixados novamente.
- **Conjuntos (Sets):**
Utiliza o tipo 'set' para buscas de IDs com alta performance.
- **Robustez:**
O tratamento de exceção evita erros na primeira execução do script.

≡ Persistência em arquivo CSV local.

Script de coleta - Função principal

script.py

Função `main()`: Loop de Coleta

```
# Itera sobre intervalos de tempo
for start, end in generate_date_intervals(start, end, type):
    log("interval", f"[{start} - {end}]")

# Persistência do progresso no CSV
with open("files/atual_date.csv", "w") as csvfile:
    writer = csv.DictWriter(csvfile, fieldnames=[ "y", "m", "d" ])
    writer.writerow({
        "year": end.year,
        "month": end.month,
        "day": end.day
    })
```

Processamento:

1. Iteração:

Gera intervalos customizados (semanais/mensais) para coleta.

2. Monitoramento:

Registra via log o intervalo atual sendo processado.

3. Salvamento:

Atualiza o CSV para permitir a retomada em caso de falha.

 Persistência iterativa do estado.

Script de coleta - Função principal

script.py

Função `main()`: Verificação por vídeo

```
#Só faz a coleta do vídeo se o título possuir keywords...
for index, item in enumerate(videos, start=1):
    VIDEO_TITLE = item['snippet']['title'].lower()
    key_words = config['key_words']

    # Verifica se o título possui as palavras chave
    if any((word.lower() in VIDEO_TITLE for word in key_words) or
        len(key_words) == 0):
        video_id = item['id']['videoId']

        # Verifica se o vídeo já foi processado
        if video_id not in processed_videos:
            video_details = get_video_details(video_id)
```

Critérios de Filtragem:

- **Palavras-chave:**
O título do vídeo deve conter termos definidos na configuração.
- **Não Processado:**
O script verifica o conjunto de IDs para evitar duplicidade.
- **Coleta Ativa:**
Apenas vídeos válidos disparam a busca de detalhes

🔧 Otimização de processamento e armazenamento.

Script de coleta - Função principal

script.py

Função `main()`: Execução do Processamento

```
# Se não processado, captura detalhes e processa
if video_id not in processed_videos:
    # Detalhes e contagem de comentários
    video_details = get_video_details(video_id)
    count = video_details['comment_count']
    if count > 0:
        process_video(video_id, "", processed_videos)

log("search", f"Coleta concluída: {query}")
```

Lógica de Processamento:

1. Unicidade:

Garante que cada ID de vídeo seja tratado apenas uma vez.

2. Validação:

Verifica a existência de comentários antes de disparar a coleta.

3. Finalização:

Registra o encerramento do ciclo para a consulta atual.



Gestão eficiente de dados.

Dúvidas?

Créditos

Richardy R. Tanure. Aluno do curso de Engenharia da Computação da UFOP/ICEA.

Autor do código original de coleta de dados no Youtube.